Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ОТЧЁТ
по лабораторной работе №5
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Взаимодействие React-приложения с сервером

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 Схема взаимодействия 3
3.2 Сервис API 4
3.3 Загрузка списка 5
3.4 Добавление, редактирование и удаление 7
3.5 Поиск на сервере с debounce 9
3.6 Загрузка по частям 10
3.7 Проверка вводимых значений 11
3.8 Проверка работы 12
ЗАКЛЮЧЕНИЕ 13
ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 14
ПРИЛОЖЕНИЕ А Листинг файла api.js 15
ПРИЛОЖЕНИЕ Б Листинг файла useDiagrams.js 18
ПРИЛОЖЕНИЕ В Листинг файла App.jsx 24
ПРИЛОЖЕНИЕ Г Листинг файла validation.js 25
ПРИЛОЖЕНИЕ Д Файл пределов limits.json 27
ПРИЛОЖЕНИЕ Е Листинг файла cors.js 28

1 ЦЕЛЬ РАБОТЫ

Освоить отправку HTTP-запросов из React-приложения к REST API, обработку состояний загрузки, ошибки и успеха, оптимистичное обновление интерфейса и откат при сбое. Клиентская часть проекта DSLKit, веб-редактора диаграмм как кода из направления SaaS, подключается к серверу из лабораторных работ 1–3 с аутентификацией по токену JWT и становится основой будущего курсового проекта.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Клиент и сервер. Браузер запрашивает данные у сервера по протоколу HTTP, а сервер отвечает JSON и кодом состояния. Запрос с другого адреса браузер разрешает только тогда, когда сервер передал заголовки CORS.

Fetch и axios. `fetch` встроен в браузер и возвращает промис, который отклоняется только при сбое сети. Библиотека axios отклоняет промис и при кодах 4xx и 5xx, разбирает JSON сама и поддерживает перехватчики. Поэтому в работе выбран axios.

Состояния запроса. Каждый запрос проходит три состояния: загрузка, успех и ошибка. Интерфейс показывает индикатор в первом, данные во втором и сообщение с кнопкой повтора в третьем.

Оптимистичное обновление. Интерфейс меняется сразу, не дожидаясь ответа сервера. Если сервер отказал, изменение откатывается и пользователь видит сообщение.

Отмена запроса. Объект `AbortController` прерывает запрос, когда компонент исчез или текст поиска изменился, поэтому устаревший ответ не перезапишет свежий.

3 ХОД РАБОТЫ

3.1 Схема взаимодействия

Приложение состоит из двух частей. Сервер из лабораторной работы 3 слушает порт 3000, хранит диаграммы в базе и проверяет токен. Клиент на Vite работает на порту 5178 и обращается к серверу через модуль `api.js`. Адрес сервера задаётся переменной `VITE_API_URL` в файле `.env`. Чтобы браузер разрешил запросы, сервер получил небольшое middleware CORS. Оно сравнивает заголовок `Origin` запроса с одним разрешённым адресом клиента и называет этот адрес в ответе, а для чужого адреса не добавляет разрешающих заголовков. Заголовок `Vary: Origin` отправляется всегда, поэтому общий кэш не отдаст ответ одного адреса другому. Предварительные

запросы получают ответ 204 без проверки токена. Модуль приведён в приложении Е.

Последовательность такая. Пользователь входит, сервер возвращает токен, клиент сохраняет его в `localStorage`, и перехватчик axios подставляет его в заголовок `Authorization` каждого запроса. Если сервер отвечает кодом 401, токен удаляется и показывается форма входа с сообщением о том, что сессия истекла.

Внешний вид приложения взят из Части 1 проекта: шапка с логотипом и переключателем темы, подвал, кнопки, карточки записей и поля форм подключены из таблицы стилей сайта. Одна главная кнопка на экране, элементы управления не ниже 44 пикселей, текст не мельче 14 пикселей, у полей есть видимые подписи, а ссылка пропуска навигации ведёт к основному содержимому.

Вход и регистрация показаны на рисунке 1.

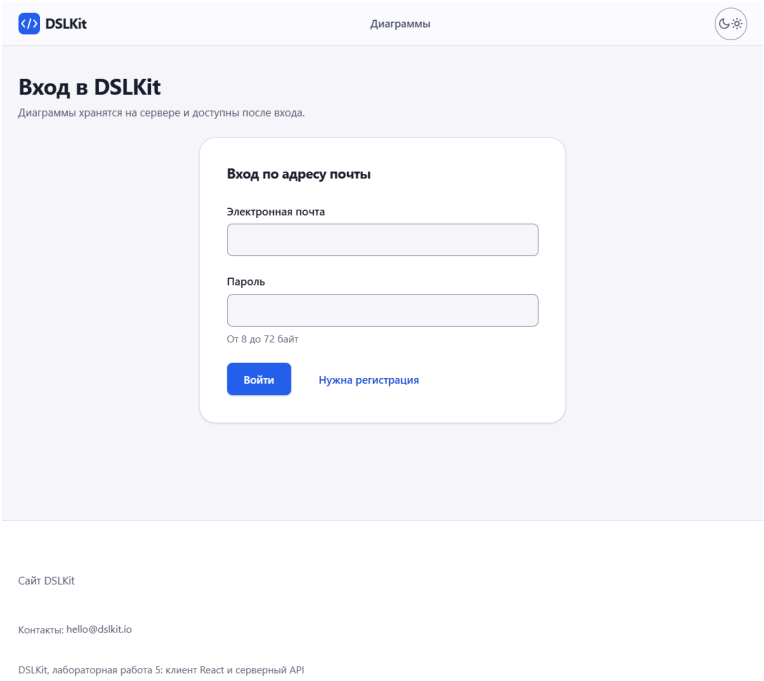


Рисунок 1 – Форма входа в приложение с проверкой адреса почты и длины пароля

3.2 Сервис API

Все обращения к серверу собраны в модуле `api.js`, который показан в приложении А. Он создаёт экземпляр axios с адресом из окружения и тайм-аутом восемь секунд, добавляет перехватчик запроса с токеном и перехватчик ответа для кода 401. Функции `fetchDiagrams`, `addDiagram`, `updateDiagram` и

`deleteDiagram` соответствуют запросам GET, POST, PUT и DELETE. Функция `errorMessage` превращает ошибку в понятный текст: сообщение сервера с перечнем полей, пояснение о тайм-ауте или сообщение о недоступном сервере.

3.3 Загрузка списка

Хук `useDiagrams` хранит список, признак загрузки и ошибку, а при монтировании запрашивает данные в `useEffect`. Хук приведён в приложении Б. Пока запрос выполняется, показывается индикатор, как на рисунке 2.

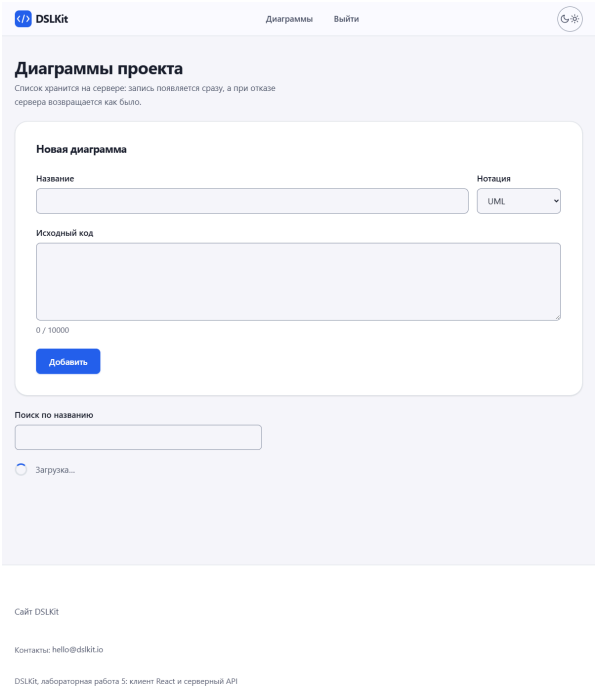


Рисунок 2 – Индикатор загрузки на медленном ответе сервера

Первый запрос просит не весь список, а одну страницу: `limit=20&offset=0`. Размер всего списка сервер отдаёт в заголовке `X-Total-Count`, а хук хранит его вместе с числом загруженных записей. Загруженный список показан на рисунке 3. У каждой записи есть поле «Создана» с точным временем, поэтому записи визуально различаются.

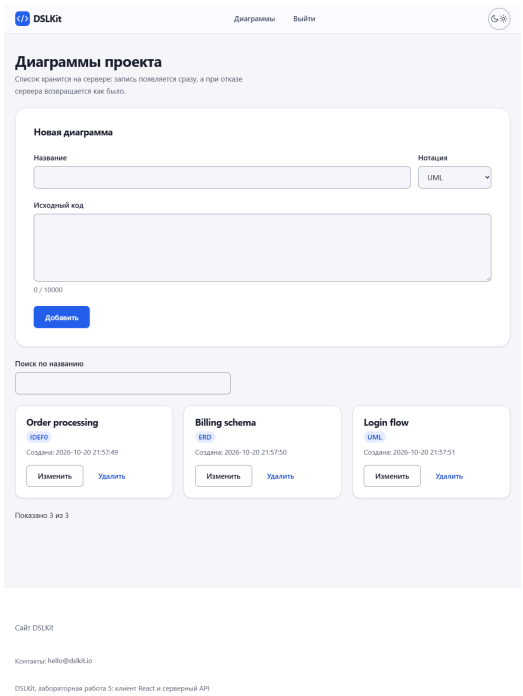


Рисунок 3 – Список диаграмм, полученный с сервера

Если сервер недоступен, вместо списка выводится сообщение и кнопка «Повторить», которая запускает запрос заново. Это состояние показано на рисунке 4.

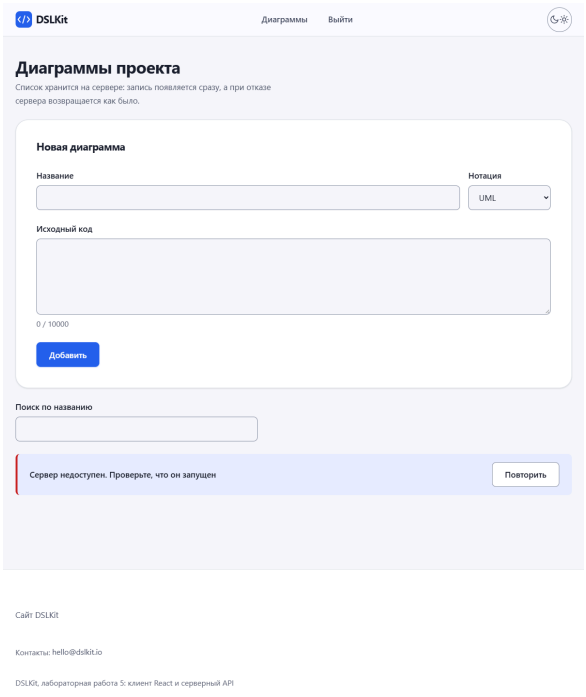


Рисунок 4 – Сообщение об ошибке при недоступном сервере и кнопка повтора

Под сообщением показан блок с разбором ошибки из трёх частей: что это, почему так вышло и как это исправить. Тексты лежат в одном каталоге `src/errorCatalog.js`, а страницу выбирает функция `kindOfFailure`: отсутствие ответа означает сбой сети, коды 401, 403 и 404 имеют свои описания, коды 5xx описаны как сбой на сервере. Для кодов вроде 400 и 409 общее описание не нужно, потому что сообщение сервера уже точное. Если падает сам интерфейс, граница ошибок показывает страницу 500, а если не удалось скачать кусок кода, страницу «Нет связи с сервером» с кнопкой повтора.

3.4 Добавление, редактирование и удаление

При добавлении запись сразу появляется в списке с временным идентификатором и пометкой «Сохраняется…». Форма и список до отправки показаны на рисунке 5.

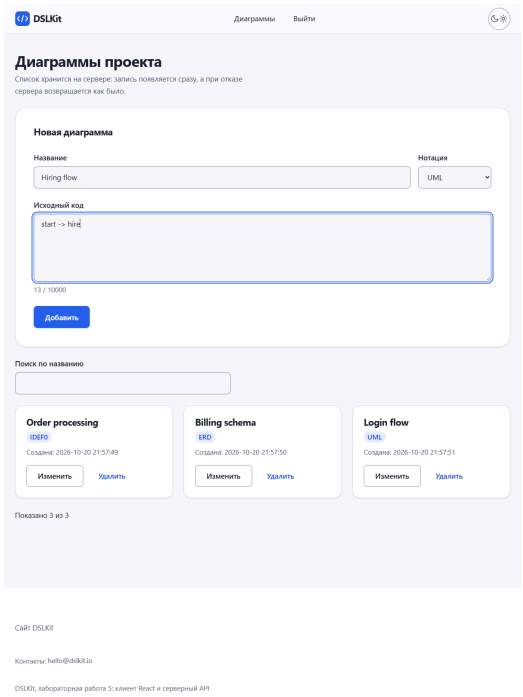


Рисунок 5 – Заполненная форма добавления перед отправкой

Сразу после нажатия запись уже в списке, хотя сервер ещё не ответил. Это видно на рисунке 6.

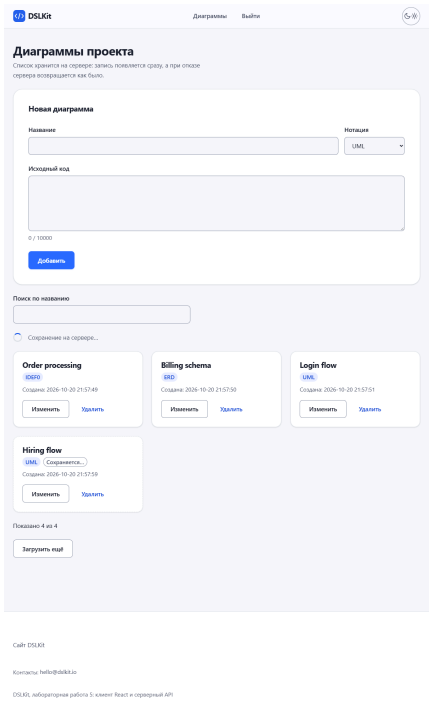


Рисунок 6 – Новая запись с пометкой «Сохраняется…» до ответа сервера

Когда сервер ответил, временная запись заменяется настоящей с идентификатором из ответа, как на рисунке 7. Если сервер отказал, запись исчезает и появляется уведомление с причиной.

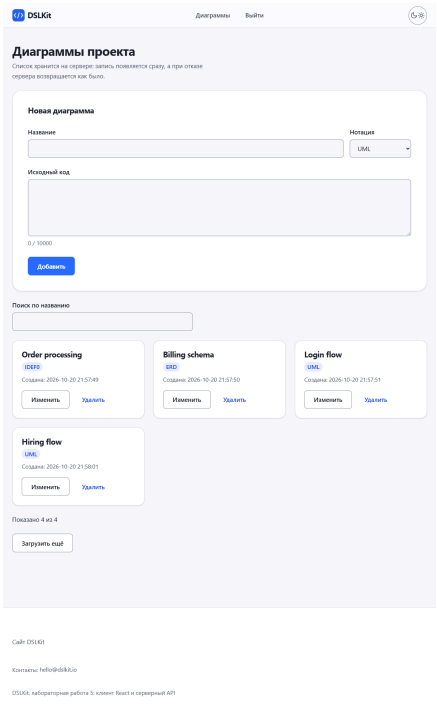


Рисунок 7 – Новая запись после ответа сервера

Удаление работает так же. Запись убирается сразу и возвращается на прежнее место, если запрос не удался. Результат показан на рисунке 8.

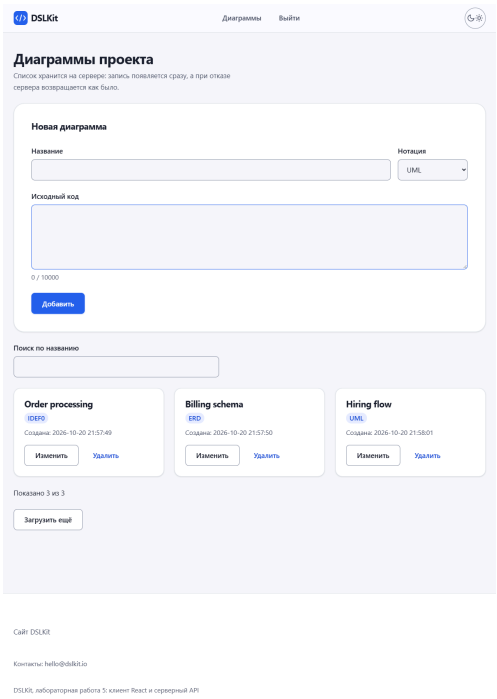


Рисунок 8 – Список после удаления записи

Редактирование названия идёт по клавише Enter или кнопке «Сохранить», а Escape отменяет правку. Клиент отправляет запрос PUT со всеми полями записи, потому что сервер заменяет запись целиком. Если запрос не удался, прежнее название возвращается.

Быстрые повторные действия не ломают список. Пока запрос по записи не завершён, второе удаление или правка этой же записи игнорируются, поэтому двойной щелчок по кнопке «Удалить» отправляет один запрос DELETE. Запись с временным идентификатором нельзя удалить или изменить, пока сервер не вернул настоящий идентификатор. Ответ 404 при удалении означает, что записи уже нет, и откат не нужен.

3.5 Поиск на сервере с debounce

В качестве дополнительного улучшения выбран поиск на сервере. Сервер принимает параметр `search` в запросе `GET /diagrams` и возвращает записи, в названии которых есть эта строка. Значение передаётся как параметр запроса к базе, поэтому внедрить в запрос чужой фрагмент нельзя, а не строка или слишком длинное значение получают ответ 400.

Клиент не отправляет запрос на каждую букву. Хук `useDebouncedValue` ждёт 400 миллисекунд тишины, а предыдущий запрос отменяется через `AbortController`. Результат поиска показан на рисунке 9.

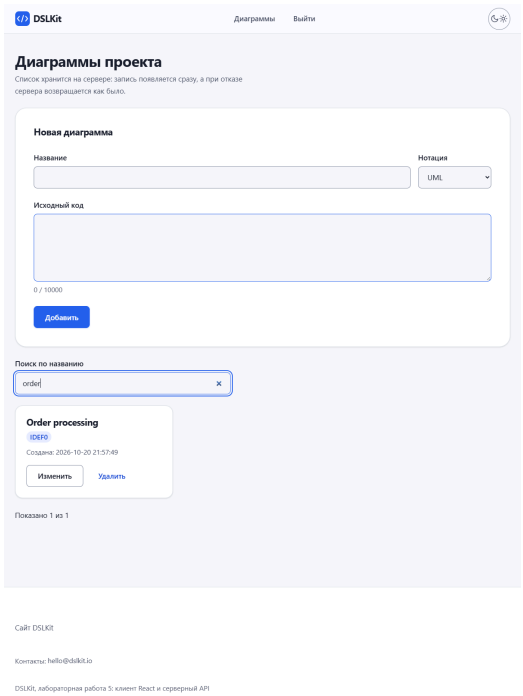


Рисунок 9 – Список после поиска по слову «order»

3.6 Загрузка по частям

Сервер умеет отдавать список страницами через параметры `limit` и `offset`, поэтому клиент показывает первые 20 записей, строку «Показано N из M» и кнопку «Загрузить ещё», как на рисунке 10. Нажатие запрашивает следующую страницу с отступом, равным числу уже загруженных записей, и дописывает её в конец списка без сброса. Пока страница грузится, кнопка заблокирована и показывает «Загрузка…», поэтому двойной щелчок отправляет один запрос. Запись, добавленная в этом сеансе, не сдвигает отступ следующей страницы, а удаление загруженной записи уменьшает его, поэтому записи не пропускаются и не дублируются. Если сервер не прислал размер списка, кнопка не показывается. Смена текста поиска отменяет загрузку страницы для прежнего поиска, и запоздавший ответ не затирает новый список. Ошибка загрузки страницы показывается уведомлением, список остаётся на месте, а кнопка работает снова.

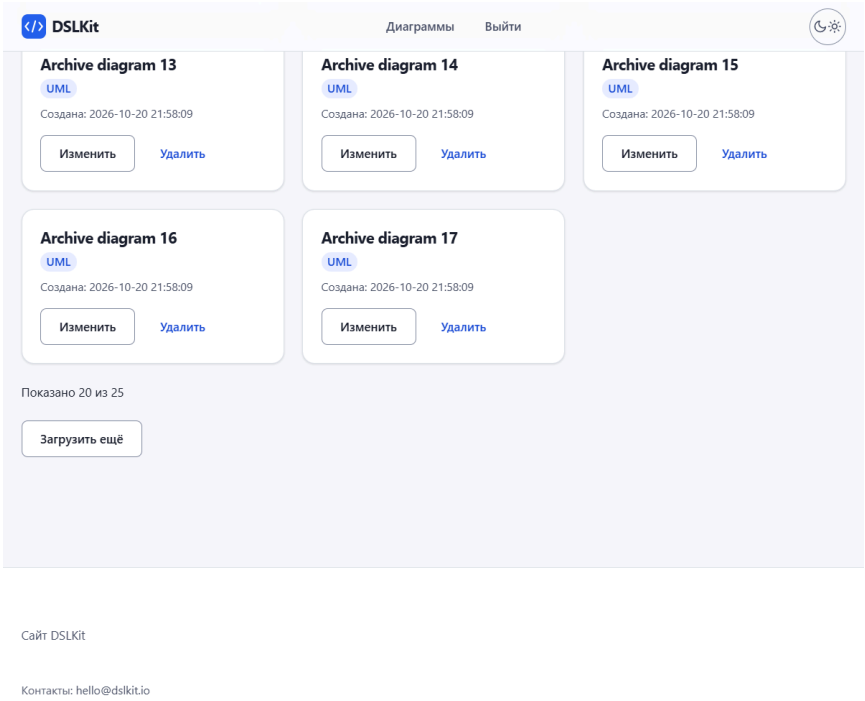


Рисунок 10 – Первая страница списка со счётчиком и кнопкой «Загрузить ещё»

Пока выполняется правка или удаление, карточка получает пометку «Сохраняется…» и `aria-busy`, её кнопки блокируются, а над списком показывается строка «Сохранение на сервере…» с индикатором, которая исчезает после ответа.

Чтение списка повторяется до трёх раз с нарастающей паузой, если сеть недоступна или сервер ответил кодом 5xx. Ответ 4xx и отменённый запрос не повторяются, а запросы на запись и удаление не повторяются никогда, чтобы не создать запись дважды. В режиме правки карточки можно изменить и нотацию, на сервер уходит запись целиком.

Главный компонент `App` оставлен коротким: он только выбирает между формой входа и списком. Состояние входа вынесено в хук `useSession`, экран списка в компонент `DiagramsScreen`, а повторяющиеся части в маленькие компоненты `Alert`, `Progress`, `SearchField` и `LoadMore`.

3.7 Проверка вводимых значений

Форма проверяет значения до отправки. Название и исходный код не могут быть пустыми или состоять из пробелов, название не длиннее 80 символов, исходный код не длиннее 10000, пароль от 8 до 72 байт. У полей названия и исходного кода есть атрибут `maxLength`, исходный код вводится в многострочное поле со счётчиком символов, а проверка остаётся на случай

обхода атрибута. Длина пароля считается в байтах, потому что хеш bcrypt на сервере учитывает только первые 72 байта, а русская буква занимает два байта. Пределы лежат в файле `limits.json` и совпадают с пределами сервера. Файл показан в приложении Д, а функции проверки в приложении Г. Сервер повторяет ту же проверку, поэтому обойти клиент не получится.

3.8 Проверка работы

Основной компонент и его состояния показаны в приложении В. Работа проверена 123 автоматическими тестами Vitest с имитацией сервера. Тесты покрывают состояния загрузки и ошибки, повтор, оптимистичное добавление, удаление и правку с откатом, поиск с отменой предыдущего запроса, вход и выход, заголовок с токеном и обработку кода 401, двойной щелчок по удалению, загрузку по частям (двойной щелчок, ошибка страницы, запоздавший ответ прежнего поиска, дубли, смещение отступа), индикатор правки и удаления, предел и счётчик многострочного поля, записи с временным идентификатором и вывод разметки из ответа сервера как обычного текста. Серверная часть с CORS и поиском проверена десятью тестами в лабораторной работе 3. Для ручной проверки нужно запустить сервер командой `npm run dev` в папке `part-2/lab-03-auth` и клиент той же командой в папке `part-2/lab-05-react-api`. Скриншоты рисунков 1–10 снимает скрипт `shots/shots.mjs`.

Ветка репозитория: 25.

ЗАКЛЮЧЕНИЕ

Клиент на React работает с серверным API: получает список с индикатором загрузки и кнопкой повтора, добавляет, правит и удаляет записи с оптимистичным обновлением и откатом, подставляет токен в каждый запрос и ищет на сервере с отложенным запросом и отменой и загружает список по частям кнопкой «Загрузить ещё». Все обращения собраны в одном модуле, а пределы значений вынесены в отдельный файл. Работа даёт готовую основу для интеграции клиента в курсовой проект.

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

9. **Как добавить заголовок Authorization в axios?** Через перехватчик: `api.interceptors.request.use((config) => { config.headers.Authorization = 'Bearer ' + token; return config; })`. В проекте токен берётся из `localStorage`.
10. **Что такое debounce?** Это задержка вызова до паузы в событиях. Для поиска запрос уходит через 400 миллисекунд после последней буквы, а не на каждую. В проекте это хук `useDebouncedValue`.
11. **Как отменить запрос при размонтировании?** Создать `AbortController` в `useEffect`, передать `signal` в запрос и вызвать `controller.abort()` в функции очистки эффекта. Так сделан хук `useDiagrams`.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг файла api.js

```
import axios from 'axios';
import limits from './limits.json';

export const TOKEN_KEY = 'token';

// The token lives in localStorage so that it survives a reload.
Storage can be blocked by the browser, then the user simply has
to sign in again.
export const getToken = () => {
  try {
    return localStorage.getItem(TOKEN_KEY);
  } catch {
    return null;
  }
};
export const saveToken = (token) => {
  try {
    localStorage.setItem(TOKEN_KEY, token);
  } catch {
    /* the session lasts until the page is closed */
  }
};
export const clearToken = () => {
  try {
    localStorage.removeItem(TOKEN_KEY);
  } catch {
    /* nothing to remove */
  }
};

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL
|| 'http://localhost:3000', timeout: 8000 });

// Every request carries the JWT saved after login.
api.interceptors.request.use((config) => {
  const token = getToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// An expired or rejected token ends the session: the token is r
emoved and the app is told to show the login form.
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 && getToken()) {
```

```
      clearToken();
      window.dispatchEvent(new Event('auth:logout'));
    }
    return Promise.reject(error);
  },
);

export const register = (email, password) => api.post('/auth/reg
ister', { email, password });
export const login = (email, password) => api.post('/auth/logi
n', { email, password });

// The search text, the page size and the offset are query param
eters (the server reads ?search=, ?limit= and ?offset=);
// the signal lets the caller cancel the request. The size of th
e whole list comes back in the X-Total-Count header.
export const fetchDiagrams = (search = '', signal, { limit, offs
et } = {}) => {
  const params = {};
  if (search) params.search = search;
  if (limit !== undefined) params.limit = limit;
  if (offset) params.offset = offset;
  return withRetry(() => api.get('/diagrams', { params: Object.k
eys(params).length > 0 ? params : undefined, signal }), signal);
};
// A read is safe to repeat, so it is tried again when the netwo
rk fails or the server answers 5xx, with a growing pause.
// A 4xx answer or a cancelled request is final: repeating would
give the same result or work against the user.
export const retryOptions = { attempts: limits.retry.attempts, d
elayMs: limits.retry.delayMs };
const isRetryable = (error) => (error.response ? error.response.
status >= 500 : !axios.isCancel(error));
const pause = (ms, signal) =>
  new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason);
    const timer = setTimeout(resolve, ms);
    signal?.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(signal.reason);
    });
    return undefined;
  });
export const withRetry = async (request, signal) => {
  for (let attempt = 1; ; attempt += 1) {
    try {
      return await request();
    } catch (error) {
      if (attempt >= retryOptions.attempts || !isRetryable(erro
r) || signal?.aborted) throw error;
      await pause(retryOptions.delayMs * attempt, signal);
    }
```

```
  }
};

export const addDiagram = (diagram) => api.post('/diagrams', dia
gram);
export const updateDiagram = (id, diagram) => api.put(`/diagram
s/${id}`, diagram);
export const deleteDiagram = (id) => api.delete(`/diagrams/${id}
`);

export const isCanceled = (error) => axios.isCancel(error);

// A text for the user: the server message when there is one, a
clear text when the server is unreachable.
// The text is shown by React as plain text, so markup in a serv
er message cannot run.
export const errorMessage = (error) => {
  if (error.response) {
    const data = error.response.data;
    const { error: message, details } = data && typeof data ===
'object' ? data : {};
    const extra = Array.isArray(details) ? details.map((d) => `
${d.field}: ${d.message}`).join(', ') : '';
    return [typeof message === 'string' && message ? message : `
Ошибка сервера (${error.response.status})`, extra].filter(Boolea
n).join(' — ');
  }
  if (error.code === 'ECONNABORTED') return 'Сервер не ответил в
овремя';
  return 'Сервер недоступен. Проверьте, что он запущен';
};

export default api;
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Листинг файла useDiagrams.js

```
import { useCallback, useEffect, useRef, useState } from 'reac
t';
import { addDiagram, deleteDiagram, errorMessage, fetchDiagrams,
isCanceled, updateDiagram } from '../api';
import { kindOfFailure } from '../errorCatalog';
import limits from '../limits.json';

const isTemporary = (id) => typeof id === 'string' && id.startsW
ith('temp-');

// The server sends the size of the whole list in X-Total-Count.
A missing or broken value means "unknown", then there is no next
page.
const readTotal = (response) => {
  const value = Number(response.headers?.['x-total-count']);
  return Number.isInteger(value) && value >= 0 ? value : null;
};

// Ids of records with a PUT or DELETE in flight, shown as a pro
gress indicator.
// A second delete or edit of the same record is ignored until t
he first one ends.
function useBusyIds() {
  const [busy, setBusy] = useState(() => new Set());
  const busyIds = useRef(new Set());
  const markBusy = (id) => {
    busyIds.current.add(id);
    setBusy(new Set(busyIds.current));
  };
  const unmarkBusy = (id) => {
    busyIds.current.delete(id);
    setBusy(new Set(busyIds.current));
  };
  return { busy, isBusy: (id) => busyIds.current.has(id), markBu
sy, unmarkBusy };
}

// Loads the list page by page: the first page for a search tex
t, then "load more".
// The request is cancelled when the search text changes, on ret
ry and when the component unmounts.
// A page that is being loaded for the old search is cancelled t
oo, so its answer never reaches the new list.
function usePagedList(search, onNotice) {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  const [loadingMore, setLoadingMore] = useState(false);
  const [error, setError] = useState(null);
  // Which explained error page fits the failure (see errorCatal
og.js), null when the server message is enough.
  const [errorKind, setErrorKind] = useState(null);
  const [attempt, setAttempt] = useState(0);
  // fetched: how many records of the server list are loaded (th
e offset of the next page), total: the size of the whole list.
  const [paging, setPaging] = useState({ fetched: 0, total: null
});
  // The latest values for the handlers, so a fast second click
never works with an old copy.
  const itemsRef = useRef(items);
  itemsRef.current = items;
  const pagingRef = useRef(paging);
  pagingRef.current = paging;
  // Records added in this session: they are not part of the loa
ded pages, so they do not move the offset of the next page.
  const localIds = useRef(new Set());
  const moreController = useRef(null);
  const loadingMoreRef = useRef(false);

  const changeTotal = (delta) => setPaging((current) => (curren
t.total === null ? current : { ...current, total: Math.max(0, cu
rrent.total + delta) }));

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setLoadingMore(false);
    loadingMoreRef.current = false;
    setError(null);
    setErrorKind(null);
    fetchDiagrams(search, controller.signal, { limit: limits.pag
eSize, offset: 0 })
      .then((response) => {
        if (controller.signal.aborted) return;
        // A body that is not a list (HTML from a proxy, an empt
y answer) is an error, it must not reach the list and crash it.
        if (!Array.isArray(response.data)) {
          setError('Сервер вернул неожиданный ответ. Проверьте а
дрес сервера');
          setErrorKind('network');
          return;
        }
        localIds.current.clear();
        setItems(response.data);
        setPaging({ fetched: response.data.length, total: readTo
tal(response) });
      })
      .catch((failure) => {
        if (!isCanceled(failure) && !controller.signal.aborted)
```

```
{
          setError(errorMessage(failure));
          setErrorKind(kindOfFailure(failure));
        }
      })
      .finally(() => {
        if (!controller.signal.aborted) setLoading(false);
      });
    return () => {
      controller.abort();
      moreController.current?.abort();
    };
  }, [search, attempt]);

  const retry = useCallback(() => setAttempt((n) => n + 1), []);

  // "Load more": the next page is appended, the records already
on the screen are never doubled.
  // A second click while a page is loading is ignored.
  const loadMore = useCallback(async () => {
    const { fetched, total } = pagingRef.current;
    if (loadingMoreRef.current || total === null || fetched >= t
otal) return;
    loadingMoreRef.current = true;
    setLoadingMore(true);
    const controller = new AbortController();
    moreController.current = controller;
    try {
      const response = await fetchDiagrams(search, controller.si
gnal, { limit: limits.pageSize, offset: fetched });
      if (controller.signal.aborted) return;
      const loaded = response.data;
      loaded.forEach((record) => localIds.current.delete(record.
id));
      setItems((current) => {
        const known = new Set(current.map((item) => item.id));
        return [...current, ...loaded.filter((record) => !known.
has(record.id))];
      });
      setPaging({ fetched: fetched + loaded.length, total: readT
otal(response) ?? total });
    } catch (failure) {
      if (!isCanceled(failure) && !controller.signal.aborted) on
Notice(`Не удалось загрузить ещё. ${errorMessage(failure)}`);
    } finally {
      if (!controller.signal.aborted) {
        loadingMoreRef.current = false;
        setLoadingMore(false);
      }
    }
  }, [search, onNotice]);
```

```
  return { items, setItems, itemsRef, paging, setPaging, changeT
otal, localIds, loading, loadingMore, error, errorKind, retry, l
oadMore };
}

// Owns the server state of the list: the pages plus the optimis
tic add, delete and edit.
export function useDiagrams(search) {
  const [notice, setNotice] = useState(null);
  const tempCounter = useRef(0);
  const { busy, isBusy, markBusy, unmarkBusy } = useBusyIds();
  const list = usePagedList(search, setNotice);
  const { items, setItems, itemsRef, paging, setPaging, changeTo
tal, localIds } = list;

  const dismissNotice = useCallback(() => setNotice(null), []);

  // Optimistic add: the item appears at once with a temporary i
d, the real record replaces it on success.
  const add = useCallback(async (fields) => {
    tempCounter.current += 1;
    const tempId = `temp-${tempCounter.current}`;
    setItems((current) => [...current, { ...fields, id: tempId,
createdAt: new Date().toISOString(), pending: true }]);
    changeTotal(1);
    try {
      const { data } = await addDiagram(fields);
      localIds.current.add(data.id);
      setItems((current) => current.map((item) => (item.id === t
empId ? data : item)));
      return true;
    } catch (failure) {
      setItems((current) => current.filter((item) => item.id !==
tempId));
      changeTotal(-1);
      setNotice(`Не удалось добавить диаграмму. ${errorMessage(f
ailure)}`);
      return false;
    }
  }, []);

  // Optimistic delete: the item disappears at once and returns
to its place when the server refuses.
  // A record that is still being created (temporary id) or alre
ady being changed is skipped.
  const remove = useCallback(async (id) => {
    const index = itemsRef.current.findIndex((item) => item.id =
== id);
    if (index < 0 || isTemporary(id) || isBusy(id)) return;
    const removed = itemsRef.current[index];
    const wasLocal = localIds.current.delete(id);
    markBusy(id);
```

```
    setItems((current) => current.filter((item) => item.id !== i
d));
    changeTotal(-1);
    if (!wasLocal) setPaging((current) => ({ ...current, fetche
d: Math.max(0, current.fetched - 1) }));
    try {
      await deleteDiagram(id);
    } catch (failure) {
      // 404 means the record is already gone, so the list is ri
ght and nothing is restored.
      if (failure.response?.status !== 404) {
        setItems((current) => (current.some((item) => item.id ==
= id) ? current : [...current.slice(0, index), removed, ...curre
nt.slice(index)]));
        changeTotal(1);
        if (wasLocal) localIds.current.add(id);
        else setPaging((current) => ({ ...current, fetched: curr
ent.fetched + 1 }));
        setNotice(`Не удалось удалить диаграмму. ${errorMessage
(failure)}`);
      }
    } finally {
      unmarkBusy(id);
    }
  }, []);

  // Optimistic edit: PUT replaces the whole record, so the serv
er gets all fields; the old record comes back on failure.
  const update = useCallback(async (id, changes) => {
    const previous = itemsRef.current.find((item) => item.id ===
id);
    if (!previous || isTemporary(id) || isBusy(id)) return fals
e;
    const { title, notation, sourceCode, priority } = { ...previ
ous, ...changes };
    markBusy(id);
    setItems((current) => current.map((item) => (item.id === id
? { ...item, ...changes } : item)));
    try {
      const { data } = await updateDiagram(id, { title, notatio
n, sourceCode, priority });
      setItems((current) => current.map((item) => (item.id === i
d ? data : item)));
      return true;
    } catch (failure) {
      setItems((current) => current.map((item) => (item.id === i
d ? previous : item)));
      setNotice(`Не удалось сохранить изменения. ${errorMessage
(failure)}`);
      return false;
    } finally {
      unmarkBusy(id);
```

```
    }
  }, []);

  const hasMore = paging.total !== null && paging.fetched < pagi
ng.total;
  // Requests that are still running in the background: shown as
one progress line for edits, deletes and adds.
  const syncing = busy.size + items.filter((item) => item.pendin
g).length;

  return {
    items,
    loading: list.loading,
    loadingMore: list.loadingMore,
    error: list.error,
    errorKind: list.errorKind,
    notice,
    busy,
    syncing,
    hasMore,
    total: paging.total,
    retry: list.retry,
    dismissNotice,
    loadMore: list.loadMore,
    add,
    remove,
    update,
  };
}
```

ПРИЛОЖЕНИЕ В
(обязательное)
Листинг файла App.jsx

```
import AuthForm from './components/AuthForm';
import DiagramsScreen from './components/DiagramsScreen';
import Footer from './components/Footer';
import Header from './components/Header';
import { useSession } from './hooks/useSession';

const TEXTS = {
  in: { title: 'Диаграммы проекта', lead: 'Список хранится на се
рвере: запись появляется сразу, а при отказе сервера возвращаетс
я как было.' },
  out: { title: 'Вход в DSLKit', lead: 'Диаграммы хранятся на се
рвере и доступны после входа.' },
};

export default function App() {
  const { signedIn, expired, logout, loggedIn } = useSession();
  const text = signedIn ? TEXTS.in : TEXTS.out;

  return (
    <div className="page">
      <a className="skip-link" href="#main">
        Перейти к основному содержимому
      </a>
      <Header signedIn={signedIn} onLogout={logout} />
      <main className="main" id="main">
        <div>
          <h1 className="page-title">{text.title}</h1>
          <p className="page-lead">{text.lead}</p>
        </div>
        {signedIn ? <DiagramsScreen /> : <AuthForm notice={expir
ed ? 'Сессия истекла. Войдите снова.' : undefined} onLoggedIn={l
oggedIn} />}
      </main>
      <Footer />
    </div>
  );
}
```

ПРИЛОЖЕНИЕ Г
(обязательное)
Листинг файла validation.js

```
import limits from '../limits.json';

export const NOTATIONS = ['UML', 'BPMN', 'ERD', 'DFD', 'IDEF0',
'Petri'];
export const PRIORITIES = ['low', 'normal', 'high'];

const checkText = (value, { minLength, maxLength }, label) => {
  const length = value.trim().length;
  if (length < minLength) return `Введите ${label}`;
  if (length > maxLength) return `${label[0].toUpperCase()}${lab
el.slice(1)} не длиннее ${maxLength} символов`;
  return null;
};

// Returns an object with a message per invalid field, empty whe
n everything is valid.
// A whitespace-only text counts as empty. The limits live in li
mits.json and match the server.
export function validateDiagram({ title, sourceCode, notation })
{
  const errors = {};
  const titleError = checkText(title, limits.title, 'название');
  const codeError = checkText(sourceCode, limits.sourceCode, 'ис
ходный код');
  if (titleError) errors.title = titleError;
  if (codeError) errors.sourceCode = codeError;
  if (!NOTATIONS.includes(notation)) errors.notation = 'Выберите
нотацию';
  return errors;
}

// The server hashes the password with bcrypt, which reads only
the first 72 bytes, so the limit counts bytes (a Cyrillic letter
is two bytes).
export function validateCredentials({ email, password }) {
  const errors = {};
  const address = email.trim();
  if (!/^\S+@\S+\.\S+$/.test(address) || address.length > limit
s.email.maxLength) errors.email = 'Введите адрес электронной поч
ты';
  if (password.length < limits.password.minLength) errors.passwo
rd = `Пароль не короче ${limits.password.minLength} символов`;
  else if (new TextEncoder().encode(password).length > limits.pa
ssword.maxBytes) errors.password = `Пароль не длиннее ${limits.p
assword.maxBytes} байт`;
  return errors;
```

```
}

// "2026-10-16 12:30:05" in local time, the same look in every l
ocale; every record shows its own creation time.
export const formatTime = (iso) => new Date(iso).toLocaleString
('sv-SE');
```

ПРИЛОЖЕНИЕ Д
(обязательное)
Файл пределов limits.json

```
{
  "title": {
    "minLength": 1,
    "maxLength": 80
  },
  "sourceCode": {
    "minLength": 1,
    "maxLength": 10000
  },
  "email": {
    "maxLength": 254
  },
  "password": {
    "minLength": 8,
    "maxBytes": 72
  },
  "searchDebounceMs": 400,
  "pageSize": 20,
  "retry": {
    "attempts": 3,
    "delayMs": 300
  }
}
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Листинг файла cors.js

```
// A browser app on another origin (the React client) may call t
he API only when the server allows that origin.
// origin: the single allowed origin such as http://localhost:51
78. It is compared with the Origin header of the request:
// only an equal value gets the Access-Control-Allow-* headers,
every other origin gets none and the browser blocks the answer.
// Vary: Origin is always sent so a shared cache never gives the
answer for one origin to another.
// A browser preflight (OPTIONS with Access-Control-Request-Meth
od) is answered with 204 here and never reaches the token check.
// A plain OPTIONS request without that header goes on to the ro
utes, which answer it with the Allow header.
const createCors = (origin) => {
  if (!origin || origin === '*') throw new Error('corsOrigin mus
t be one exact origin, not * (see .env.example)');
  return (req, res, next) => {
    res.vary('Origin');
    const allowed = req.get('Origin') === origin;
    if (allowed) {
      res.set('Access-Control-Allow-Origin', origin);
      // Lets the browser app read the total size of a paged lis
t.
      res.set('Access-Control-Expose-Headers', 'X-Total-Count');
    }
    if (req.method !== 'OPTIONS' || !req.get('Access-Control-Req
uest-Method')) return next();
    if (allowed) {
      res.set('Access-Control-Allow-Methods', 'GET, POST, PUT, P
ATCH, DELETE, OPTIONS');
      res.set('Access-Control-Allow-Headers', 'Content-Type, Aut
horization');
      res.set('Access-Control-Max-Age', '600');
    }
    return res.status(204).end();
  };
};

module.exports = { createCors };
```